

1. Comprensión del capítulo

El capítulo 4 del material **HCIP Big Data Developer V2.0 (2022)** se centra en el *procesamiento de flujos de datos en tiempo real* («Big Data Real-Time Stream Processing»). Describe las motivaciones, arquitecturas y tecnologías clave para **ingestar y analizar datos en movimiento** (Flume, Kafka, Flink, Spark Structured Streaming, Redis), así como ejemplos de uso como la detección de fraude en tarjetas y análisis de e-commerce en tiempo real. En conjunto, aborda cómo procesar continuamente eventos de alta velocidad para generar *insights* inmediatos.

En detalle, el capítulo enfatiza que el **problema resuelto por el procesamiento en tiempo real** es el de extraer valor de datos de alta velocidad en el momento en que llegan. A diferencia del procesamiento por lotes (que analiza datos históricos acumulados), el **stream processing** permite reaccionar inmediatamente ante eventos nuevos. Esto es crucial en escenarios operativos sensibles, donde un retraso en el análisis puede significar oportunidades perdidas o riesgos mayores ¹ ². Por ejemplo, en fraude con tarjetas de crédito es necesario evaluar transacciones en milisegundos para bloquear pagos sospechosos antes de que se completen ³ ⁴.

El capítulo contrasta claramente **procesamiento por flujo** (stream processing) con **procesamiento off-line por lotes y recuperación en tiempo real**. En el enfoque *batch*, los datos se almacenan y procesan periódicamente (por ejemplo, una vez al día), sacrificando inmediatez a favor de exhaustividad. En cambio, *stream processing* opera sobre cada evento a medida que llega, buscando baja latencia. La “recuperación en tiempo real” podría interpretarse como la capacidad de consultar o reaccionar inmediatamente a los resultados actualizados (por ejemplo, búsquedas interactivas o consultas a sistemas en memoria), pero no implica procesamiento complejo, sino más bien uso en línea de datos recientes. El flujo de datos continuo exige un modelo de cómputo diferente que atenúe la demora que sufren los lotes. Como resultado, se obtienen «analítica en tiempo real» y capacidad de respuesta instantánea a sucesos críticos ¹ ⁵.

Finalmente, el capítulo identifica varios **requisitos de una plataforma de streaming de gran escala**. Entre ellos se destacan: **latencia** ultra-baja, para procesar eventos en milisegundos; **alto throughput**, para manejar miles o millones de eventos por segundo; **confiabilidad y tolerancia a fallos**, de modo que el sistema siga funcionando pese a caídas de nodos o de red; **escalabilidad horizontal**, para crecer añadiendo máquinas; **ingesta de múltiples fuentes**, soportando conectividad con distintos orígenes de datos (sensores, logs, bases de datos, redes sociales, etc.); y **aislamiento de recursos**, para permitir multi-tenencia o diferentes cargas en paralelo. En resumen, la plataforma debe soportar entrega garantizada de eventos (al menos una vez o exactamente una vez), checkpointing del estado, manejo de eventos fuera de orden y **mecanismos de corrección ante fallos** ⁶ ⁷. En la literatura técnica se subraya que en streaming los indicadores clave son la latencia de procesamiento y el throughput sostenido ⁸ ⁹, junto con la capacidad de recuperar estado consistente ante caídas ¹⁰ ¹¹.

2. Marco conceptual profundo

El **flujo de datos** (o *data stream*) es una secuencia continua de *eventos* o registros que se generan a lo largo del tiempo. Un **evento** es típicamente un objeto con información (por ejemplo, un JSON) que describe una ocurrencia en un instante particular (transacción de compra, lectura de sensor, clic en un sitio web, etc.). El **procesamiento continuo** implica aplicar lógica (filtros, agregaciones, modelos) al

flujo a medida que llega, en lugar de esperar a acumularlo. Esto es crítico porque en muchas aplicaciones el tiempo es una dimensión esencial: el orden temporal de los eventos y su marca de tiempo determinan el significado de los patrones observados ¹² ¹³ .

Una distinción clave es entre **tiempo de evento** (*event time*) y **tiempo de procesamiento** (*processing time*). El *tiempo de evento* es la hora registrada en el evento cuando ocurrió en el mundo real (por ejemplo, la marca del reloj del dispositivo origen) ¹⁴ . El *tiempo de procesamiento* es el momento en que el sistema procesa el evento (relacionado con el reloj de la máquina). Además existe el *tiempo de ingestión* (hora en que el evento ingresa al broker). Usar el tiempo de evento permite análisis temporales precisos (reconstruir la línea de tiempo real), mientras que usar el tiempo de procesamiento es más sencillo pero puede sesgar resultados cuando hay retrasos o desorden. Las **ventanas de tiempo** (time windows) son intervalos sobre los que se agrupan eventos para agregaciones (por ejemplo, cuentas por minuto). Hay varios tipos: *ventanas fijas/tumbling* (intervalos contiguos sin solapamiento), *deslizantes/sliding* (solapadas, cada cierto intervalo), *de sesión* (basadas en inactividad entre eventos) y *instantáneas* (agrupan por marca de tiempo exacta) ¹⁵ ¹⁶ . Las ventanas permiten realizar cálculos por períodos temporales, pero requieren decidir cuándo cerrarlas y cómo manejar eventos tardíos.

Para procesar streams de manera fiable es necesario mantener **estado**. El *estado* es información acumulada en el sistema (por ejemplo, contadores, ventanas actuales, sumas parciales) que influye en el procesamiento de nuevos eventos ¹⁷ . El sistema debe guardar este estado de forma tolerante a fallos. Aquí entran los **checkpoints**, que son instantáneas consistentes periódicas del estado global (incluyendo la posición en la fuente de datos) ¹⁰ . En caso de fallo, el procesador puede restaurar el estado desde el último checkpoint y reempezar el cómputo sin pérdida de consistencia.

El tratamiento de eventos *fuera de orden* es otro desafío: en entornos reales, un evento con marca de tiempo antigua puede llegar después de otros más recientes. Para gestionarlo se usan **watermarks** (marcas de agua): secuencias especiales que “avanzan” el reloj de evento del sistema. Un watermark con timestamp t indica que ya no se esperan eventos con tiempo menor a t . Flink, por ejemplo, usa watermarks para disparar el cierre de ventanas de event-time y detectar eventos tardíos ¹⁸ . Según la documentación, “un watermark declara que hasta ese punto en el tiempo se han recibido todos los datos relevantes” ¹² . Esto permite al sistema saber cuándo producir resultados (por ejemplo, cerrar una ventana) incluso con retrasos de red y eventos desordenados.

Respecto a la **exactitud en la entrega**, hay distintas garantías: *at-least-once* y *exactly-once*. *At-least-once* asegura que cada evento **se procesa al menos una vez**, pero puede producir duplicados en caso de reenvío tras fallo ¹⁹ . Para tolerar duplicados se requiere que las escrituras de salida sean *idempotentes* (la misma entrada aplicada varias veces produce el mismo efecto) ²⁰ . En contraste, *exactly-once* asegura que cada evento afecta el resultado **exactamente una vez** (sin pérdidas ni duplicados observables). En la práctica esto se logra coordinando la fuente, el procesamiento y el sumidero mediante checkpointing y dos fases de commit ¹¹ . Tal como describen referencias recientes, *exactly-once* implica más complejidad operativa y latencia, de modo que a menudo se opta por *at-least-once* junto con sinks idempotentes para balancear fiabilidad y rendimiento ²¹ ²⁰ .

En resumen, el procesamiento de streams requiere concebir el tiempo como dimensión crítica (event-time vs processing-time), aplicar ventanas para agrupar eventos, gestionar *estado* con mecanismos de *checkpoint*, y establecer semánticas de entrega (exactly-once vs at-least-once) apoyadas en idempotencia. Estos conceptos (tiempo de evento, ventanas, estado, checkpoints, watermarks, orden/desorden, garantías de entrega) son fundamentales para diseñar sistemas fiables de flujo de datos ¹⁸ ¹¹ .

3. Arquitectura tecnológica

Una **arquitectura típica de streaming extremo a extremo** consta de varias capas: (1) *fuentes* de datos, (2) capa de *ingesta/broker*, (3) motor(es) de *procesamiento en tiempo real*, (4) capas de *almacenamiento/visualización* y (5) *actuadores* o salidas operativas.

- **Fuentes de datos (Sources):** pueden ser sensores IoT, logs de aplicaciones, transacciones bancarias, redes sociales, etc. Estas fuentes generan eventos continuamente. Por ejemplo, en el caso de fraude con tarjetas, la fuente son terminales de pago o APIs bancarias que emiten transacciones.
- **Ingesta/Broker:** aquí entran herramientas como **Apache Flume** y **Apache Kafka**. Flume está diseñado para la recolección eficiente de **logs y datos en streaming**: tiene agentes que leen archivos o flujos y los envían a canales intermedios (a menudo HDFS, Kafka, o bases) ²². Es común usar Flume para cenar logs de servidores o sensores y volcar a Kafka o HDFS. Kafka, en cambio, es una plataforma de mensajería distribuida de alta capacidad: actúa como un *log persistente* basado en topics-particiones, permitiendo publicar/consumir datos en tiempo real ²³. Kafka garantiza alta durabilidad y throughput, escalando horizontalmente. Como dice la documentación, “Kafka habilita ingesta de datos tolerante a fallos y de alto rendimiento con distribución basada en topics” ²³. A menudo Flume se configura para escribir directamente a Kafka, combinando la fácil recolección de Flume con la robusta distribución de Kafka ²² ²⁴.
- **Procesamiento:** las tecnologías clave incluyen **Apache Flink** y **Spark Structured Streaming** (además de Kafka Streams). Flink es un motor de procesamiento *nativo por streams*, con soporte completo de event-time, ventanas y estado. Es ideal para casos de uso críticos en tiempo real, pues maneja streams no acotados con latencia mínima ²⁵ ¹⁷. Como señala Conductor, Flink es “un framework de streaming distribuido de código abierto diseñado para manejar streams infinitos con baja latencia y alto throughput” ²⁵. Su API de DataStream (o SQL/Table) permite control de estado y semánticas exactly-once. En contraste, Spark Structured Streaming extiende Spark para streaming usando *micro-batches* ²⁶. Usa el mismo API de DataFrame/Dataset; por defecto agrupa datos en mini-lotes (cada pocos centenares de ms), aunque desde Spark 3.2 ofrece modo continuo experimental ²⁶. Spark es más intuitivo para equipos ya familiarizados con Spark/Python, y permite integrarse con ML de Spark. Sin embargo, presenta latencias típicas en el orden de los cientos de ms a segundos ²⁷, mientras que Flink opera por evento con latencias sub-segundo ²⁷. Ambos motores soportan ventanas de event-time, pero Flink ofrece un manejo más avanzado de eventos fuera de orden mediante *watermarks* nativos ²⁸ ¹⁸.
- **Almacenamiento y sinks:** tras el procesamiento, los resultados pueden escribirse en bases de datos, caches o sistemas de mensajería. **Redis** se usa a menudo como almacén en memoria para datos dinámicos o estados compartidos. Por ejemplo, tras procesar flujos de sesiones web, un sistema puede almacenar en Redis los recuentos de clics por usuario, habilitando consultas muy rápidas. Redis también ofrece *Streams* (estructura de datos) que puede servir de cola o buffer. Otras salidas típicas son bases de datos NoSQL (Cassandra, MongoDB), data warehouses (Snowflake, BigQuery), o dashboards en tiempo real (Grafana conectado a TimescaleDB, por ejemplo).

Comparativa de tecnologías:

- *Apache Flume* se especializa en **ingesta distribuida de logs y datos brutos**. Posee componentes *Source – Channel – Sink* y puede recolectar datos de múltiples servidores hacia destinos como

HDFS o Kafka ²². Es fiable y escalable, pero su lógica de procesamiento es básica (no hace agregaciones complejas).

- *Apache Kafka* es un **sistema de mensajería distribuido** que funciona como bus de eventos. Guarda datos en log persistente dividido en particiones, lo que permite a muchos consumidores leer en paralelo. Resuelve la ingesta masiva (alta capacidad) y garantiza tolerancia a fallos por replicación de particiones. Kafka, a diferencia de Flume, ofrece relectura de eventos históricos (útil para re-procesamientos) y conecta con ecosistema (Kafka Streams, Kafka Connect para CDC).
- *Apache Flink* es un **motor de procesamiento de streams de propósito general**, ideal para cálculos de alta complejidad en tiempo real. Soporta estado manejado (persistencia de estado en backends como RocksDB), semánticas exactly-once integradas y ventanas avanzadas ²⁹ ¹⁰. Es más complejo de configurar, pero maneja muy bien casos de uso de negocio crítico (fraude, monitoreo, et al.).
- *Spark Structured Streaming* es un **framework basado en Spark SQL** para streaming. Usa un modelo de micro-batch (a menos que se active el modo continuo), lo que lo hace cercano al procesamiento por lotes pero con baja latencia. Ofrece API SQL/DataFrame familiares, integrando queries estructuradas en pipelines streaming ²⁶. Su ventaja es la facilidad de uso (especialmente para analistas/ingenieros de datos) y la integración con Spark ML. La desventaja es latencia ligeramente mayor y dependencias de Spark (requiere cluster de Spark).
- *Redis* (en contexto de streaming) **actúa como almacén de baja latencia**. Puede usarse como sink para resultados de streaming (ej. guardar agregaciones de ventanas) o como almacén de estado externo para consultas rápidas (por ejemplo, conservar conteos de eventos en tiempo real). Redis permite lecturas/escrituras en milisegundos, pero no está diseñado para procesamiento complejo sino como base de datos clave-valor de alta velocidad.

Estas tecnologías se **integran** en pipeline de ejemplo:

Imaginemos un flujo de transacciones bancarias: cada terminal emite un evento JSON con datos de la compra. Un agente Flume recoge estos logs de varios servidores de punto de venta y los envía a un *topic* de Kafka. Kafka persiste estos eventos y los distribuye a consumidores. Un clúster de Flink suscrito a ese *topic* procesa cada transacción: aplica reglas de detección de fraude usando ventanas de minutos para detectar múltiples compras inusuales. Los eventos marcados como sospechosos se escriben en Redis (por su baja latencia) para consulta inmediata por un servicio de monitoreo o API. Además, otro job de Spark Structured Streaming puede leer el mismo *topic* de Kafka para alimentar dashboards de analítica de ventas en tiempo real. Finalmente, las decisiones operativas (bloqueo de tarjeta, alerta) se basan en las señales producidas. En este flujo, **Flume** facilita la recolección; **Kafka** desacopla los componentes y almacena el log de eventos; **Flink/Spark** llevan a cabo la lógica de análisis; y **Redis** almacena resultados intermedios para visualización o acciones.

4. Estado del arte y ampliación externa

Además del contenido del capítulo, diversas fuentes recientes aportan perspectivas actualizadas:

- **Kafka moderno:** Según Apache, desde Kafka 3.5 en adelante *ZooKeeper está marcado como obsoleto* y **Kafka 4.0 (marzo 2025)** eliminó completamente el soporte de ZooKeeper ³⁰ ³¹. La

nueva arquitectura *KRaft* integra el control de metadatos en el propio cluster, mejorando la operatividad y eliminando una capa externa. Esto ha permitido reducciones de costos y mejoras de rendimiento (por ejemplo, réplicas más rápidas y escalado simplificado) ³⁰ ³¹ . Otros avances de Kafka 4.0 incluyen un nuevo protocolo de rebalanceo de consumidores más eficiente (KIP-848) y la introducción de colas nativas KIP-932 para comunicación punto a punto. En la práctica, esto significa que las implementaciones actuales deben planificar la migración a KRaft (Kafka sin ZooKeeper) para beneficiarse de mejoras de estabilidad y rendimiento ³⁰ ³¹ .

- **Apache Flink actualizado:** Flink 1.18 (2023) y sucesivas versiones han introducido mejoras notables. Por ejemplo, se agregó un controlador JDBC para la *SQL Gateway*, lo que permite usar clientes SQL estándar para interactuar con tablas de Flink ³² . También se incorporaron funciones de *stored procedures* en conectores, permitiendo encapsular lógica compleja de bases externas dentro de Flink ³³ . Estos avances reflejan que Flink avanza hacia ser una plataforma de *streaming lakehouse*, integrándose con ecosistemas SQL/BI. Flink 1.19 y posteriores siguen mejorando el rendimiento de su backend RocksDB, el escalado, y la integración con Kubernetes. Además, la comunidad promueve el uso de *PyFlink* y Flink ML para facilitar su uso en entornos Python y de machine learning. (Estos detalles provienen de documentación oficial de Apache Flink ³² y blogs técnicos.)
- **Spark Structured Streaming actual:** En los últimos años Spark 3.x ha reforzado su modo de *procesamiento continuo* además del micro-batch tradicional. Databricks presentó el “Real-time Mode” que promete latencias de milisegundos en Structured Streaming ²⁶ . No obstante, en la mayoría de casos Spark Structured sigue operando por micro-batches de ~500ms-1s ²⁷ . Las fuentes actuales destacan que Structured Streaming *garantiza semánticas exactly-once* mediante relectura de fuentes replayables y sink idempotentes ³⁴ ²¹ . A nivel de ecosistema, Spark integra cada vez más con Delta Lake para procesar streams directamente en tablas ACID, y con MLlib para pipelines de machine learning en tiempo real. Estos aspectos amplían lo presentado en el capítulo con prácticas de última generación.
- **Alternativas de ingesta y arquitecturas:** Más allá de Flume y Kafka, existen soluciones modernas como Apache Pulsar, Amazon Kinesis o Google Pub/Sub para ingesta de eventos. Las fuentes recomiendan considerar Pulsar (multi-tenant y con segmentación dinámica) como alternativa a Kafka en nuevos proyectos. En cuanto a patrones de arquitectura, se destaca el **patrón Kappa** para simplificar pipelines (stream único en lugar de dual batch+stream) ³⁵ . Según artículos recientes, Kappa es adecuado cuando la misma lógica puede aplicarse tanto a datos históricos como en tiempo real, evitando duplicar código como en Lambda ³⁵ . El capítulo menciona el procesamiento en tiempo real, y hoy se recomienda combinarlo frecuentemente con *arquitecturas híbridas* (p. ej. capa batch para corrección periódica y capa streaming para velocidad) o adopción de Kappa cuando es viable ³⁵ .

En resumen, las fuentes externas muestran que las ideas del capítulo (Flume, Kafka, Flink, Spark) siguen vigentes pero han evolucionado. **Parte** de la información (como el rol de Flume/Kafka) proviene del capítulo, mientras que **parte** de estos avances (Kafka sin ZooKeeper, Flink 1.18, Spark real-time) proviene de documentación oficial y blogs actualizados. Se recomienda al lector contrastar siempre el estado actual de las herramientas con el contenido del material de 2022. Por ejemplo, mientras el capítulo enfatiza Flume, las prácticas modernas apuntan a Kafka Connect para ingesta, y mientras se discute Spark Structured, hoy está emergiendo el procesamiento continuo en Spark y su integración con tecnologías lakehouse.

5. Casos de uso y aplicaciones

El procesamiento en tiempo real se aplica en numerosos dominios. A continuación se describen ejemplos reales, indicando el **evento de entrada, el procesamiento realizado, la salida/decisión y la importancia de la baja latencia**. Se relacionan con los ejemplos del capítulo (fraude y e-commerce) y otros sectores:

- **Detección de fraude bancario (tarjetas de crédito):**

- *Evento:* Cada transacción de tarjeta (JSON con ID de tarjeta, monto, ubicación, etc.) enviada desde el POS o app bancaria.
- *Procesamiento:* Un motor en Flink o Spark Streaming evalúa cada transacción mediante un modelo de riesgo. Por ejemplo, agrega en ventanas cortas los patrones de gasto por tarjeta y aplica reglas (viajes simultáneos a diferentes países, compras atípicas, combinaciones de eventos previos como varios intentos de PIN fallido). También puede inferir contexto (¿cambió IP o dispositivo?). Se utilizan ventanas de event-time para correlacionar eventos cercanos.
- *Salida:* Si se detecta posible fraude, el sistema emite una señal: bloquear temporalmente la transacción, congelar la cuenta o alertar al cliente por móvil ³ ⁴. También registra el evento en un log de auditoría.
- *Importancia de baja latencia:* Fundamental para minimizar pérdidas. Debe bloquearse o confirmar transacciones en milisegundos: si se tarda horas, el dinero ya se perdió. Como muestran casos reales, la prevención en tiempo real puede reducir drásticamente las pérdidas por fraude ³. Este caso aparece en el capítulo de Huawei (fraude con tarjetas).

- **Análítica e-commerce en tiempo real (personalización y recomendaciones):**

- *Evento:* Los clics, vistas o compras que hace un usuario en un sitio web. Cada acción genera un evento (userID, itemID, timestamp, etc.).
- *Procesamiento:* Un pipeline de Spark Structured Streaming o Flink agrupa eventos en ventanas de segundos o minutos por usuario. Calcula métricas como número de clics por categoría, páginas vistas recientes, o patrones de compra. Puede alimentar un motor de recomendación (por ejemplo, un modelo colaborativo). Se actualizan perfiles de usuarios en Redis o base rápida.
- *Salida:* Se generan recomendaciones personalizadas instantáneamente (productos sugeridos) o actualizaciones de tablero de control. También se pueden ajustar precios dinámicamente en función de demanda.
- *Importancia de baja latencia:* Crucial para la experiencia de usuario. Una recomendación basada en la acción que el usuario acaba de realizar es mucho más relevante que una obsoleta. Estudios señalan que la relevancia y conversión aumentan si la personalización es verdaderamente en tiempo real ³⁶. El capítulo menciona análisis e-commerce en tiempo real, y aquí se concreta cómo un clic reciente influye al instante.

- **Internet de las Cosas (IoT) – Monitoreo de sensores industriales:**

- *Evento:* Lecturas de sensores (temperatura, presión, vibración, etc.) enviadas continuamente por máquinas en una planta.
- *Procesamiento:* Un flujo de Spark Streaming o Flink agrupa lecturas por máquina en pequeñas ventanas y calcula estadísticas (promedio, desviación). Se puede aplicar detección de anomalías: por ejemplo, comparar la lectura actual con umbrales históricos o modelos predictivos en tiempo real.

- *Salida:* Si se detecta una condición anómala (por ejemplo, temperatura excesiva o variación inesperada), el sistema dispara alertas de mantenimiento inmediato. También se actualizan tableros de control con estado de equipo en tiempo real.
- *Importancia de baja latencia:* Permite reaccionar antes de que ocurra un fallo grave. Detectar una falla inminente al segundo puede evitar pérdidas de producción o accidentes. Las empresas ya usan pipelines de streaming para *mantenimiento predictivo*, reaccionando en tiempo real ante datos de IoT ³⁷.

• **Monitoreo de sistemas / DevOps:**

- *Evento:* Métricas de servidores (CPU, memoria) y logs de aplicaciones en tiempo real.
- *Procesamiento:* Un sistema como Flink consume estos eventos, calcula tasas de errores o alertas en ventanas cortas, y aplica umbrales.
- *Salida:* Genera alertas automatizadas (por ejemplo, notifica un fallo en el servicio o escala instancias en la nube).
- *Importancia de baja latencia:* Ayuda a los equipos de TI a detectar y resolver incidentes inmediatamente, minimizando tiempos de inactividad.

• **Ciberseguridad / Detección de intrusiones:**

- *Evento:* Logs de red, intentos de login, tráfico de firewall o IDS.
- *Procesamiento:* Un pipeline de streaming correlaciona eventos sospechosos. Por ejemplo, podría detectar intentos repetidos de acceso (eventos de login fallidos) seguidos de un acceso exitoso en poco tiempo, o patrones de escaneo de puertos.
- *Salida:* En tiempo real bloquea IPs o genera alertas de seguridad al SOC. También se retroalimenta a firewalls o SIEM.
- *Importancia de baja latencia:* Identificar una intrusión *mientras ocurre* reduce daños. Si se demorara, el atacante ya podría exfiltrar datos. Los S.I.E.M. modernos utilizan streaming para correlación en tiempo real ³⁸.

• **Telecomunicaciones – Control de calidad de red:**

- *Evento:* Eventos de red como registros de llamadas (CDR), reportes de calidad de señal, logs de base stations.
- *Procesamiento:* Flink agrupa eventos por región o celda en ventanas, detectando congestión o caídas. También correlaciona métricas de uso masivo con calidad de servicio.
- *Salida:* Ajuste dinámico de recursos (por ejemplo, añadir nodos o redistribuir carga), o envío de alertas a técnicos.
- *Importancia de baja latencia:* Permite corregir problemas de red al vuelo y mantener la calidad de servicio, esencial para evitar clientes insatisfechos.

• **Sistemas de recomendación en medios o retail:**

- *Evento:* Eventos de navegación o consumo de contenido (vídeos vistos, música escuchada, productos comprados).
- *Procesamiento:* Similar al e-commerce, pero en contextos de streaming de contenido. Se actualizan perfiles de usuario y rankings de contenido en tiempo real.

- **Salida:** Listas de reproducción personalizadas, banners dinámicos, notificaciones push con sugerencias.
- **Importancia de baja latencia:** Ofrece experiencias personalizadas en el momento justo, aumentando engagement y ventas ³⁶.

En cada caso, la **salida** suele ser una acción automatizada (bloqueo, alerta, ajuste) o una visualización en paneles que permite intervención humana. En todos ellos la **baja latencia** es crítica: como explica la literatura, la diferencia de unos segundos puede transformar decisiones operativas—por ejemplo, evitar transacciones fraudulentas o enviar alertas antes de una falla mayor ³ ⁴.

6. Material utilizable para mis entregables

Para apoyar la monografía, presentación y laboratorios, destacamos los conceptos y estructuras clave:

- **Conceptos imprescindibles (monografía):**

- *Flujo de datos (stream) y evento.*
- *Procesamiento continuo vs batch.*
- *Latencia, throughput, escalabilidad, confiabilidad, tolerancia a fallos.*
- *Time semantics:* Tiempo de evento, tiempo de procesamiento, tiempo de ingestión.
- *Ventanas (tumbling, sliding, sesión, instantánea).*
- *Estado y checkpoints.*
- *Watermarks y eventos tardíos.*
- *Orden/desorden de eventos.*
- *Semánticas de entrega:* at-least-once, exactly-once, idempotencia.
- *Tecnologías clave:* Flume, Kafka, Flink, Spark Structured Streaming, Redis (con sus roles y límites).
- *Patrones arquitectónicos:* Lambda vs Kappa, pipelines de capas (speed, batch).
- *Casos de uso:* Detalle de ejemplos reales (fraude, IoT, personalización, etc.).
- *Herramientas complementarias:* Kafka Connect, Kafka Streams, Pulsar, Beam (según alcance).
- *Contexto actual:* Tendencias recientes (Kafka 4.0 KRaft, Flink SQL, Spark real-time).

- **Estructura sugerida para presentación (10–15 diapositivas):**

- **Título:** “Procesamiento de Streams en Tiempo Real a Gran Escala” (tema, autor, contexto).
- **Motivación / Problema:** Diferencia entre *batch* vs *stream*, por qué es necesario el procesamiento en tiempo real ¹ ⁵.
- **Requisitos clave:** Latencia, throughput, escalabilidad, tolerancia a fallos (mencionar brevemente cada uno) ⁸ ⁶.
- **Conceptos fundamentales:** Definir flujo, evento, event-time vs processing-time ¹⁴ ³⁹.
- **Ventanas y estado:** Explicar tipos de ventanas y concepto de estado/checkpoint con íconos o diagramas simplificados.
- **Delivery Semantics:** At-least-once vs exactly-once (tablas o listados) ²¹.
- **Arquitectura típica:** Diagrama general con fuentes, Flume/Kafka, Flink/Spark, sink (incl. Redis/BD) – breve anotación de funciones.
- **Tecnologías: Flume vs Kafka:** Bullet: roles y diferencias ²² ²³.
- **Tecnologías: Flink vs Spark Streaming:** Comparativa (tabla o puntos clave) ²⁵ ²⁷.
- **Redis y otras herramientas:** Ejemplos de uso (cache, state store).
- **Caso de uso 1 – Fraude:** Flujo de datos y decisiones (diagrama/flujo) ³ ⁴.
- **Caso de uso 2 – E-commerce:** Flujo y KPIs (ilustrar recomendaciones en tiempo real) ³⁶.
- **Otros casos breves:** IoT, ciberseguridad, telecom (listado rápido).

- **Tendencias y mejores prácticas:** Kafka KRaft, Flink 1.18+, Spark real-time, Kappa architecture

30 35 .

- **Conclusiones/Recomendaciones:** Recapitulación de beneficios y desafíos; pasos futuros (ej. adoptar pruebas en prototipo, seleccionar framework adecuado, etc.).

- **Propuestas de laboratorios en código (Python/Kafka/PySpark/Flink):**

- **Laboratorio 1 – Contador simple con Spark Structured Streaming (difícil baja):**

- *Objetivo:* Familiarizarse con Spark Structured Streaming leyendo datos en tiempo real y aplicando una agregación.
- *Herramientas:* Python, PySpark, Kafka (o sockets TCP).
- *Entrada:* Secuencia de líneas de texto o eventos simples (por ejemplo, líneas con palabras) provenientes de Kafka o de un socket (simulado con `nc`).
- *Procesamiento:* Spark lee el stream (lectura continua), cuenta ocurrencias de una palabra clave o eventos por ventana de 10 segundos (ventana tumbling).
- *Salida:* Imprime en consola o guarda en un sink (por ejemplo, un archivo o Redis) los conteos por ventana.
- *Dificultad: Fácil.* Requiere instalar PySpark y crear un tópico Kafka (o usar socket). Sirve de base para comprender flujos de datos, JSON/strings, y ventanas básicas.

- **Laboratorio 2 – Detección de anomalías simples con Flink y Kafka (media):**

- *Objetivo:* Implementar un pipeline de Flink que detecta eventos fuera de rango en un flujo de datos (p.ej. transacciones elevadas).
- *Herramientas:* Python (o Java) con PyFlink o Flink Python API, Kafka local.
- *Entrada:* Flujo de eventos JSON simulando transacciones bancarias (campos: tarjeta, monto, timestamp), publicados periódicamente en un tópico Kafka.
- *Procesamiento:* Flink consume el tópico, aplica un `KeyBy` por tarjeta y calcula una ventana deslizante (sliding window) de 1 minuto que verifica el monto promedio o la suma. Si excede un umbral, marca como anomalía. Además utiliza *watermarks* para tolerar retrasos en el envío de eventos (se puede simular retrasos con `sleep`).
- *Salida:* Los eventos etiquetados (normales o anomalía) se escriben en otro tópico Kafka o en Redis. Mostrar resultado (por ejemplo, imprimir casos de anomalía).
- *Dificultad: Media.* Involucra instalar Kafka, configurar Flink (modo local), programar ventanas y manejar estado. Aprenderá sobre detección sencilla en streaming y garantías (Flink ofrece checkpoint por defecto).

- **Laboratorio 3 – Pipeline completo de ecommerce con Spark Streaming y Redis (difícil):**

- *Objetivo:* Construir un pipeline de ingestión y análisis en tiempo real con múltiples tecnologías integradas.
- *Herramientas:* Python, Kafka, PySpark Structured Streaming, Redis. Posible uso de Docker Compose para levantar Kafka y Redis.
- *Entrada:* Simulación de eventos de e-commerce en formato JSON (userID, productoID, categoría, timestamp), enviados desde un script Python hacia Kafka a velocidad variable.
- *Procesamiento:* Spark Structured Streaming consume el tópico de Kafka, realiza uniones de estado (por ejemplo, agrupa por userID), calcula métricas (p. ej. frecuencia de visita a categoría en última hora con *stateful processing*). Usa ventana sliding para agrupar. También debe lidiar con datos tardíos usando *event time* y *watermarks* (configurar en Spark).

- **Salida:** Resultados agregados (p. ej. conteos de vistas por usuario/categoría) se escriben a Redis. Además, puede actualizar una base de datos interna o generar un aviso (por ejemplo, publicar otro tópico Kafka). Un pequeño script cliente lee de Redis para mostrar métricas actualizadas en tiempo real (o un mini-dashboard en consola).
- **Dificultad: Alta.** Requiere coordinar Kafka + Spark + Redis. Ilustra un escenario realista de arquitectura lambda simplificada. El estudiante aprenderá sobre ingestión, estado en streaming, sinks y componentes adicionales. El lab debe incluir instrucciones claras y datos de ejemplo para probar los resultados.

Estos laboratorios están ordenados de menor a mayor complejidad. El primero es viable en cualquier equipo con PySpark; el segundo añade Flink (o PyFlink) para experiencia con un motor nativo de streaming; el tercero integra varios componentes para simular un caso de uso más completo. Se sugiere proveer datos de muestra o scripts de generación automática para facilitar la experimentación.

7. Referencias y calidad académica

Todas las afirmaciones técnicas aquí expuestas se basan en fuentes reconocidas, recientes y documentadas:

- **Fuentes primarias:** Documentación oficial de Apache Flink ²⁵ ³², Apache Kafka ³¹ ³⁰, Confluent (expertos en Kafka y Flink) ³ ⁴⁰, y artículos de investigación académica sobre stream processing ¹ ⁶.
- **Fuentes secundarias:** Blogs técnicos autorizados y documentos de la industria (Confluent, Conductor, Fivetran, Streamkap, etc.) ⁸ ³⁵ que, aunque no son académicos, provienen de ingeniería reconocida y aportan contexto práctico actualizado.

Se han evitado afirmaciones sin respaldo. Cuando existe discrepancia, se notifica explícitamente. Por ejemplo, el material de Huawei enfatiza herramientas como Flume; sin embargo, fuentes actuales señalan que en muchos proyectos se opta ahora por *Kafka Connect* o servicios administrados para ingesta, reduciendo el uso de Flume en nuevos desarrollos. También, mientras el capítulo estudia Spark Structured Streaming, se destaca que Spark 3.5+ soporta “Real-Time Mode” para latencias menores ²⁶. Tales diferencias ilustran la importancia de contrastar con la documentación más reciente.

Síntesis final y recomendaciones: En conjunto, el procesamiento de datos en tiempo real es esencial para aplicaciones modernas (fraude, monitoreo, personalización, IoT, ciberseguridad, etc.). Para construir los entregables, se recomienda:

1. **Monografía:** Organizarla en secciones claras cubriendo teoría (definiciones, conceptos claves), arquitectura (capas de pipeline, comparación de tecnologías) y casos prácticos. Incluir definiciones precisas (evento, ventana, estado), diagramas de flujo de datos y ejemplos reales con cifras. Incorporar las fuentes citadas para dar soporte a cada afirmación. Destacar las diferencias actuales (Kafka KRaft, Flink SQL, Spark continuous) como ampliación del material base.
2. **Presentación (10–15 diapositivas):** Usar la estructura propuesta, priorizando diagramas y ejemplos visuales. Cada diapositiva debe tener pocos puntos clave. Por ejemplo, usar un diagrama sencillo de arquitectura de streaming, o un flujo de detección de fraude con Flink/Kafka. En caso de citas, mencionarlas en el pie o como referencia compacta para credibilidad. Incluir comparativas en tablas (p.ej. Flink vs Spark) para claridad.

3. **Laboratorio:** Seleccionar el o los laboratorios adecuados al nivel (idealmente el *Lab 2* o *Lab 3* para un desafío realista). Prover guías paso a paso, con datos de muestra incluidos (JSON de transacciones o eventos). Sugerir entornos ejecutables (p.ej. notebooks, Docker Compose) y comentar las observaciones esperadas (p.ej. cómo ver los resultados en Redis). El laboratorio debe permitir al estudiante practicar con Python + Kafka + motor de streaming (PySpark o PyFlink) y ver en acción conceptos como ventanas y estado.

Con estas recomendaciones, se integrarán fundamentos teóricos, comparaciones técnicas y práctica programática en cada entrega, asegurando profundidad, claridad pedagógica y utilidad para el curso de Minería de Datos y Aprendizaje Automático.

Referencias: Las citas exactas a literatura y documentación se han incluido a lo largo del texto con el formato estándar ¹ ²⁰, garantizando trazabilidad de las afirmaciones.

¹ ⁶ ⁹ ²³ Real-time data stream processing in large-scale systems

https://wjarr.com/sites/default/files/fulltext_pdf/WJARR-2022-0903.pdf

² ⁷ ⁸ Stream Processing: An Introduction

<https://www.confluent.io/learn/stream-processing/>

³ How Real-Time Streaming Prevents Fraud in Banking & Payments

<https://www.confluent.io/blog/real-time-streaming-prevents-fraud/>

⁴ ³⁶ ³⁷ ³⁸ Stream processing guide: Real-time data and use cases

<https://www.fivetran.com/learn/stream-processing>

⁵ ²⁰ Batch vs. Real-Time Data Ingestion: Differences Explained | Unstructured

<https://unstructured.io/insights/batch-vs-real-time-data-ingestion-key-differences-explained>

¹⁰ ¹³ ¹⁷ ²⁵ ²⁹ ³⁹ What is Apache Flink? Stateful Stream Processing | Conduktor

<https://www.conduktor.io/glossary/what-is-apache-flink-stateful-stream-processing>

¹¹ ¹⁹ ²¹ Exactly-Once vs At-Least-Once: Choosing Delivery Guarantees - Streamkap

<https://streamkap.com/resources-and-guides/exactly-once-vs-at-least-once>

¹² ¹⁸ Understanding Apache Flink Event Time and Watermarks

<https://www.decodable.co/blog/understanding-apache-flink-event-time-and-watermarks>

¹⁴ Event-Time Processing

<https://developer.confluent.io/patterns/stream-processing/event-time-processing/>

¹⁵ ¹⁶ Introducción a las funciones de ventanas de Azure Stream Analytics - Azure Stream Analytics | Microsoft Learn

<https://learn.microsoft.com/es-es/azure/stream-analytics/stream-analytics-window-functions>

²² ²⁴ Apache Flume: Introduction to Data Extraction and Movement

<https://www.confluent.io/learn/apache-flume/>

²⁶ ²⁷ ²⁸ ³⁴ Spark Streaming vs Apache Flink®: Modern Stream Processing Compared

<https://www.confluent.io/compare/spark-streaming-vs-flink/>

³⁰ Kafka 4.0 Removes ZooKeeper: KRaft Migration Guide for 2026 | byteiota

<https://byteiota.com/kafka-4-zookeeper-kraft-migration-guide-2/>

³¹ ZooKeeper | Apache Kafka

<https://kafka.apache.org/37/operations/zookeeper/>

32 33 **Announcing the Release of Apache Flink 1.18 | Apache Flink**

<https://flink.apache.org/2023/10/24/announcing-the-release-of-apache-flink-1.18/>

35 **The Kappa Architecture: Simplifying Data Pipelines with Streaming - Streamkap**

<https://streamkap.com/resources-and-guides/kappa-architecture-guide>

40 **Apache Flink: Stream Processing for All Real-Time Use Cases**

<https://www.confluent.io/blog/apache-flink-stream-processing-use-cases-with-examples/>